Análise dos Resultados — Sprint 4

Construção de um LLM do Zero · Pedro Devens Dellalibera e João Gabriel Trevisol

Esta análise interpreta os cinco experimentos registrados no documento de experimentação da Sprint 4, relacionando cada alteração de arquitetura ao seu efeito sobre o número de parâmetros e sobre o custo computacional do modelo GPT implementado em `4,2.py`.

1. n_layers e o peso dos componentes fixos

O Experimento 2 mostra que cada `TransformerBlock` custa exatamente 7.085.568 parâmetros, número constante, independente de quantos blocos existem. Reduzir `n_layers` de 12 para 4 economiza 8 blocos inteiros (56.684.544 parâmetros), mas o total do modelo mini não cai para 1/3 do baseline, e sim para 65%. `tok_emb` e `out_head` somados já são 77.194.752 parâmetros — quase metade do baseline inteiro — e esses dois não dependem de `n_layers` em momento algum.

A conclusão é que, com um vocabulário de 50.257 tokens, o tamanho do vocabulário domina o orçamento de parâmetros de um modelo raso. Empilhar mais blocos só passa a pesar proporcionalmente quando o modelo já é profundo o bastante para que `n_layers` × 7.085.568 supere o custo fixo dos embeddings e da cabeça de saída. No GPT-124M esse custo fixo é de 77.982.720 parâmetros, o que coloca o ponto de equilíbrio por volta de 11 blocos — ou seja, praticamente na configuração original de 12 camadas.

2. n_heads não é um parâmetro de capacidade

O Experimento 3 confirma, na arquitetura completa, a mesma conclusão que a Sprint 3 já tinha tirado isolando apenas a `MultiHeadAttention`: o total de parâmetros não muda nem um pouco entre 12 e 4 cabeças. `W_query`, `W_key`, `W_value` e `out_proj` continuam com shape 768 × 768 de qualquer forma — o que muda é só `head_dim = emb_dim / num_heads` (de 64 para 192) e como esse mesmo espaço é reparticionado no `forward`.

Isso reforça o ponto da Sprint 3: `num_heads` é um parâmetro de particionamento, não de capacidade. Mais cabeças dão mais subespaços para aprender padrões diferentes, mas sem nenhum custo adicional de memória de pesos. O compromisso é entre diversidade de padrões e profundidade de cada subespaço, não entre tamanho e capacidade.

3. Dropout: zero parâmetros, efeito apenas em modo treino

O Experimento 4 confirma algo estrutural: `nn.Dropout` não tem peso aprendível, então `drop_rate` nunca aparece na contagem de `contar_parametros`. E como todas as medições desta sprint rodam com `modelo.eval()`, o dropout sequer chega a ser aplicado — mesma lógica já registrada na análise da Sprint 3 sobre por que os experimentos de lá usaram taxa 0,0.

A diferença entre 0.1 e 0.3 só terá efeito observável quando houver backward pass de verdade, no pré-treinamento do capítulo 5: taxas mais altas zeram uma fração maior das ativações a cada passo,

regularizando mais forte e, em excesso, atrapalhando o aprendizado. Nada disso é visível rodando apenas `forward()` num modelo sem treino.

4. context_length custa muito menos que n_layers — em parâmetros

O Experimento 5 isola exatamente o que muda com `context_length`: apenas o tamanho de `pos_emb` (`Embedding(context_length, emb_dim)`). A queda de 1024 para 256 libera 589.824 parâmetros — exatamente 768 × (1024 − 256) — contra os 56.684.544 liberados ao cortar `n_layers` de 12 para 4 no Experimento 2. Em proporção, `context_length` pesa menos de 1% do que pesou `n_layers`.

Seria tentador concluir que `context_length` é caro no outro eixo, o do processamento. Mas os tempos medidos mostram que é preciso cuidado com essa leitura. A Sprint 3 mostrou que a matriz de atenção cresce quadraticamente — só que com o número de tokens efetivamente processados, não com o limite declarado na configuração. Como a máscara é recortada com `[:num_tokens, :num_tokens]` dentro do `forward`, e o lote de teste tem apenas 6 tokens, a matriz de atenção é 6 × 6 nas duas configurações. Em outras palavras, neste experimento `context_length` não deveria alterar o tempo de forward.

A economia de 16× no custo da atenção existe, mas só se manifestaria com sequências que realmente usassem 1024 contra 256 tokens. O que o Experimento 5 isola, de fato, é apenas o eixo de memória de parâmetros — e nesse eixo o efeito é mínimo. A variação de tempo observada (76% do baseline) está dentro da margem de erro discutida no item 7.

5. Weight tying: de onde vêm os ~124M do nome do modelo

Em todas as cinco configurações, a diferença entre a soma ingênua e o total com weight tying é sempre 38.597.376 — exatamente o tamanho de `out_head` (`vocab_size` × `emb_dim`, sem bias). O GPT-2 real reusa a matriz de `tok_emb` como `out_head`, em vez de treinar uma segunda matriz gigante do zero. É por isso que o baseline bate com os 124.412.160 que dão nome ao "GPT-2 124M", e não com os 163.009.536 que `contar_parametros` soma sem essa otimização.

Vale registrar que esta implementação (`4,2.py`) ainda não compartilha os pesos de fato: ela apenas calcula o que a técnica economizaria. O weight tying de verdade — apontar `out_head.weight` para o mesmo tensor de `tok_emb.weight` — é uma mudança de código que fica em aberto para quando o treino entrar em cena, no capítulo 5.

6. Contagem de operações prevê o tempo — quando o efeito é grande

Os parâmetros são determinados inteiramente pela arquitetura, por isso os cinco valores do documento de experimentação são exatos e batem com o que o `4,2.py` já havia impresso no baseline. Essa é uma propriedade útil: dá para prever o tamanho de um modelo antes de instanciá-lo, apenas somando as camadas.

O tempo de forward, não. Ele depende da CPU, da carga da máquina no momento e da build do PyTorch. Ainda assim, dá para prever sua ordem de grandeza contando operações — e o Experimento

2 confirma isso com boa precisão. Para o lote usado (2 × 6 = 12 posições), cada Transformer Block custa 7.077.888 multiplicações-acumulações por posição, enquanto o `out_head` custa 463.168.512 operações fixas. O baseline soma cerca de 1,48 bilhão e o modelo mini cerca de 803 milhões, o que prevê 54% do tempo. O medido foi 51%.

É o mesmo fenômeno do item 1, agora no eixo do processamento: assim como o `out_head` impede que os parâmetros caiam para 1/3, ele também impede que o tempo caia para 1/3. Cortar dois terços dos blocos reduz o tempo pela metade, não a um terço, porque a projeção final sobre o vocabulário continua inteira.

7. Os controles acidentais e o que eles revelam sobre a medição

O resultado metodologicamente mais útil desta sprint não estava previsto. Os Experimentos 3 e 4 alteram parâmetros que, comprovadamente, não mudam nem o número de pesos nem a quantidade de operações do forward: `n_heads` apenas reparticiona matrizes de mesmo tamanho, e o `drop_rate` não tem efeito algum com o modelo em `eval()`. Os dois deveriam, portanto, ter reproduzido o tempo do baseline.

Deram 120% e 114%. Como sabemos, por construção, que o valor correto seria 100%, essa dispersão não mede nenhuma propriedade da arquitetura: ela mede o ruído do próprio método. Com 5 repetições num desktop comum, sujeito ao escalonamento de frequência da CPU e a outros processos do sistema, a margem de erro é da ordem de ±20%.

Isso reorganiza a leitura de todos os resultados. Apenas o Experimento 2 (51%) produziu uma diferença claramente acima do ruído — e, não por acaso, é o único cuja magnitude também era prevista pela contagem de operações. A variação do Experimento 5 (76%), que à primeira vista parecia confirmar um ganho de desempenho, tem a mesma ordem de grandeza dos controles e não sustenta conclusão alguma, como discutido no item 4.

A lição é geral, e vale para as próximas sprints: antes de atribuir significado a uma diferença de tempo, é preciso saber qual é a variação esperada quando nada muda. Sem essa referência, qualquer oscilação de medição pode ser confundida com efeito de arquitetura. Uma medição mais confiável exigiria muitas repetições e o relato da dispersão, não apenas da média.

Limitações desta etapa

Vale registrar, como já fizemos na análise da Sprint 3: todos os cinco modelos têm pesos aleatórios, sem nenhuma iteração de treino. Os números de parâmetros e as relações entre eles são conclusões arquiteturais válidas independentemente de treino — mas qualquer diferença entre os textos gerados pelas cinco configurações não tem significado nenhum nesta etapa, porque nenhum dos modelos aprendeu nada ainda.

Também vale registrar que o lote de teste tem apenas 6 tokens. Isso é suficiente para comparar o custo dos blocos e da cabeça de saída, mas pequeno demais para que o custo quadrático da atenção apareça — motivo pelo qual o Experimento 5 não pôde demonstrar o efeito que a teoria prevê para contextos longos.

O que esta sprint entrega é a arquitetura GPT completa, testada em cinco configurações plausíveis, com os custos relativos de cada parâmetro mapeados nos dois eixos: memória e processamento. O pré-treinamento do capítulo 5 é o que vai dar sentido real ao texto gerado — e é também quando o `drop_rate` do Experimento 4 passará a ter efeito mensurável.